



Universidad Nacional de Córdoba
Facultad de Ciencias Exactas, Físicas y Naturales

Sistemas de Computación
Trabajo Práctico N° 4

Docentes:
Ing. Miguel Ángel Solinas
Ing. Javier Alejandro Jorge

Autores:
Matías Lucas Morales Robles
José Ignacio Torres
Esteban Samuel Suarez

Índice

1. Introducción	2
2. Desarrollo	3
2.1. Instalación de paquetes	3
2.2. Compilación del primer módulo	3
2.3. Desafío 1	6
2.4. Desafío 2	8
2.4.1. ¿Qué funciones tiene disponible un programa y un módulo?	8
2.4.2. Espacio de Usuario vs. Espacio del Kernel	8
2.4.3. Espacio de datos	9
2.4.4. Drivers y Gestión de Periféricos en /dev	9
2.5. Preguntas para el informe	10
2.5.1. ¿Qué diferencias se pueden observar entre los dos modinfo?	10
2.5.2. ¿Qué drivers/módulos están cargados en sus propias PC?	11
2.5.3. ¿Cuáles módulos no están cargados pero están disponibles?	14
2.5.4. Correr hwinfo en una pc real con hw real y agregar la url de la información de hw en el reporte.	14
2.5.5. ¿Qué diferencia existe entre un módulo y un programa?	15
2.5.6. ¿Cómo puede ver una lista de las llamadas al sistema que realiza un simple hello- world en c?	15
2.5.7. ¿Qué es un segmentation fault? ¿Cómo lo maneja el kernel y como lo hace un programa?	16
2.5.8. Firma de un módulo de Kernel	16
2.5.9. Agregar evidencia de la compilación, carga y descarga de su propio módulo impri- miendo el nombre del equipo en los registros del kernel	17
2.5.10. ¿Que pasa si mi compañero con secure boot habilitado intenta cargar un módulo firmado por mi?	19
2.5.11. Análisis de la nota propuesta sobre Secure Boot	19

1. Introducción

Módulos del Kernel

Son fragmentos de código que permiten extender las capacidades del sistema operativo de forma dinámica, evitando la necesidad de recompilar o reiniciar el núcleo principal. Son fundamentales para añadir soporte de hardware en estructuras monolíticas sin alterar su base compleja. Dado que operan con privilegios directos, los sistemas integran mecanismos de seguridad —como firmas digitales y claves de cifrado— para garantizar que solo se ejecuten módulos autorizados y mitigar riesgos de código malicioso.

Llamadas al Sistema (System Calls)

Constituyen la interfaz primordial de interacción entre las aplicaciones y el núcleo. A través de ellas, los programas solicitan servicios esenciales como la gestión de archivos, comunicación entre procesos, asignación de memoria o control de periféricos. Este mecanismo actúa como un puente seguro que delimita el espacio de usuario del espacio del kernel, impidiendo el acceso directo a recursos críticos por razones de estabilidad. Cuando un proceso requiere una operación privilegiada, invoca una llamada específica que el núcleo ejecuta bajo un entorno estrictamente controlado.

2. Desarrollo

2.1. Instalación de paquetes

Se ejecutan los siguientes comandos:

```
1 sudo apt-get update
2 sudo apt-get install build-essential checkinstall linux-source
3 sudo apt-get install linux-headers-$(uname -r)
```

Se verifica si el sistema tiene instaladas las cabeceras del kernel actual y está correctamente preparado para compilar módulos externos:

```
1 ls /lib/modules/$(uname -r)/build
```

Los archivos fuente del proyecto se encuentran alojados en el directorio TP4/, vinculados al repositorio del grupo como un submódulo de Git:

```
1 git submodule add
   https://gitlab.com/sistemas-de-computacion-unc/kernel-modules.git
   TP4/kernel-modules
```

Así se evita duplicar el código y se mantiene la conexión con el repositorio original.

2.2. Compilación del primer módulo

A continuación se ejecutan los siguientes comandos:

```
1 cd kernel-modules
2 ls
3 cd part1
4 ls
5 cd module
6 ls
7 make
```

```
esteban@esteban-VirtualBox:~/Sistemas-de-Computacion/TP4/kernel-modules$ cd part1
esteban@esteban-VirtualBox:~/Sistemas-de-Computacion/TP4/kernel-modules/part1$ ls
module syscalls testdir
esteban@esteban-VirtualBox:~/Sistemas-de-Computacion/TP4/kernel-modules/part1$ cd module
esteban@esteban-VirtualBox:~/Sistemas-de-Computacion/TP4/kernel-modules/part1/module$ ls
Makefile mimodulo.c
esteban@esteban-VirtualBox:~/Sistemas-de-Computacion/TP4/kernel-modules/part1/module$ make
make -C /lib/modules/6.14.0-37-generic/build M=/home/esteban/Sistemas-de-Computacion/TP4/kernel-modules/part1/module
make[1]: se entra en el directorio '/usr/src/linux-headers-6.14.0-37-generic'
make[2]: se entra en el directorio '/home/esteban/Sistemas-de-Computacion/TP4/kernel-modules/part1/module'
warning: the compiler differs from the one used to build the kernel
The kernel was built by: x86_64-linux-gnu-gcc-13 (Ubuntu 13.3.0-6ubuntu2-24.04) 13.3.0
You are using:          gcc-13 (Ubuntu 13.3.0-6ubuntu2-24.04.1) 13.3.0
CC [M] mimodulo.o
mimodulo.c:8:5: warning: no previous prototype for 'modulo_lin_init' [-Wmissing-prototypes]
   8 | int modulo_lin_init(void)
     | ^~~~~~
mimodulo.c:17:6: warning: no previous prototype for 'modulo_lin_clean' [-Wmissing-prototypes]
  17 | void modulo_lin_clean(void)
     | ^~~~~~
MODPOST Module.symvers
CC [M] mimodulo.mod.o
CC [M] .module-common.o
LD [M] mimodulo.ko
BTF [M] mimodulo.ko
Skipping BTF generation for mimodulo.ko due to unavailability of vmlinux
make[2]: se sale del directorio '/home/esteban/Sistemas-de-Computacion/TP4/kernel-modules/part1/module'
make[1]: se sale del directorio '/usr/src/linux-headers-6.14.0-37-generic'
```

Como puede verse, se generó el archivo mimodulo.ko.

Ahora bien, si queremos ver una lista de los archivos que tienen extensión .ko:

```
1 ls *.ko
```

Se carga el módulo al kernel:

```
1 sudo insmod mimodulo.ko
```

Y se verifica si fue cargado:

```
1 sudo dmesg | tail
```

```
→ module git:(main) X sudo insmod mimodulo.ko
→ module git:(main) X sudo dmesg | tail
[ 8918.906575] workqueue: ata_sff_pio_task hogged CPU for >10000us 5 times, consider switchi
hing to WQ_UNBOUND
[ 8937.459616] workqueue: psi_avgs_work hogged CPU for >10000us 11 times, consider switchi
ng to WQ_UNBOUND
[ 8975.528176] workqueue: blk_mq_run_work_fn hogged CPU for >10000us 35 times, consider sw
itching to WQ_UNBOUND
[ 9006.974352] workqueue: ata_sff_pio_task hogged CPU for >10000us 7 times, consider switc
hing to WQ_UNBOUND
[ 9754.559633] workqueue: e1000_watchdog [e1000] hogged CPU for >10000us 35 times, conside
r switching to WQ_UNBOUND
[ 9923.406106] workqueue: vmstat_update hogged CPU for >10000us 7 times, consider switchin
g to WQ_UNBOUND
[10407.459405] workqueue: vmstat_shepherd hogged CPU for >10000us 4 times, consider switch
ing to WQ_UNBOUND
[10641.354300] mimodulo: loading out-of-tree module taints kernel.
[10641.354307] mimodulo: module verification failed: signature and/or required key missing
- tainting kernel
[10641.356089] Modulo cargado en el kernel.
```

Ahora se verifica que el archivo figure dentro de la lista de módulos cargados:

```
1 lsmod | grep mimodulo
```

Y se lo descarga:

```
1 sudo rmmod mimodulo
2 sudo dmesg | tail
```

```
→ module git:(main) X lsmod | grep mimodulo
mimodulo          12288  0
→ module git:(main) X sudo rmmod mimodulo
→ module git:(main) X sudo dmesg | tail
[ 8937.459616] workqueue: psi_avgs_work hogged CPU for >10000us 11 times, consider switchi
ng to WQ_UNBOUND
[ 8975.528176] workqueue: blk_mq_run_work_fn hogged CPU for >10000us 35 times, consider sw
itching to WQ_UNBOUND
[ 9006.974352] workqueue: ata_sff_pio_task hogged CPU for >10000us 7 times, consider switc
hing to WQ_UNBOUND
[ 9754.559633] workqueue: e1000_watchdog [e1000] hogged CPU for >10000us 35 times, conside
r switching to WQ_UNBOUND
[ 9923.406106] workqueue: vmstat_update hogged CPU for >10000us 7 times, consider switchin
g to WQ_UNBOUND
[10407.459405] workqueue: vmstat_shepherd hogged CPU for >10000us 4 times, consider switch
ing to WQ_UNBOUND
[10641.354300] mimodulo: loading out-of-tree module taints kernel.
[10641.354307] mimodulo: module verification failed: signature and/or required key missing
- tainting kernel
[10641.356089] Modulo cargado en el kernel.
[11359.517405] Modulo descargado del kernel.
```

Ahora ejecutamos lo siguiente para obtener información detallada acerca del módulo:

```
1 modinfo mimodulo.ko
```

```
→ module git:(main) X modinfo mimodulo.ko
filename:      /home/esteban/Sistemas-de-Computacion/TP4/kernel-modules/part1/module/mimodulo.ko
author:        Catedra de SdeC
description:    Primer modulo ejemplo
license:        GPL
srcversion:     C6390D617B2101FB1B600A9
depends:
name:          mimodulo
retpoline:     Y
vermagic:      6.14.0-37-generic SMP preempt mod_unload modversions
```

Y ejecutamos lo siguiente para comparar con un módulo oficial del kernel:

```
1 modinfo /lib/modules/$(uname -r)/kernel/crypto/des_generic.ko.zst
```

```
→ module git:(main) X modinfo /lib/modules/$(uname -r)/kernel/crypto/des_generic.ko.zst
filename:      /lib/modules/6.14.0-37-generic/kernel/crypto/des_generic.ko.zst
alias:         crypto-des3_ede-generic
alias:         des3_ede-generic
alias:         crypto-des3_ede
alias:         des3_ede
alias:         crypto-des-generic
alias:         des-generic
alias:         crypto-des
alias:         des
author:        Dag Arne Osvik <da@osvik.no>
description:    DES & Triple DES EDE Cipher Algorithms
license:        GPL
srcversion:     BA1649EB7E2F66CE23BD60D
depends:        libdes
intree:        Y
name:          des_generic
retpoline:     Y
vermagic:      6.14.0-37-generic SMP preempt mod_unload modversions
sig_id:        PKCS#7
signer:        Build time autogenerated kernel key
sig_key:       1E:D4:CC:DF:F3:2A:D3:7C:B5:68:B6:72:F3:90:D9:67:06:F8:7D:EB
sig_hashalgo:  sha512
signature:     5A:8B:D2:AF:41:03:EE:9B:59:04:AA:FC:37:7C:C7:06:7F:58:01:7E:
6F:7A:5C:8D:2A:E7:87:BC:DB:66:45:7C:6C:F9:8F:43:32:C4:A1:FF:
B9:D9:AA:2C:BA:8C:25:59:8D:6B:58:12:8A:85:F5:0D:CD:13:A7:19:
17:EE:FA:DF:D4:B8:17:A3:4F:62:56:43:1F:2D:F7:80:4A:5F:A6:6F:
AB:F4:C1:5F:75:72:14:45:92:9E:6D:17:68:6A:ED:25:DA:14:D4:3F:
26:1C:22:93:CC:64:36:A3:A1:B1:F0:DE:18:81:3F:A7:11:3C:69:F3:
45:EB:EB:3E:B5:8E:27:79:1B:27:7F:97:F9:34:7A:59:B9:59:74:A0:
6A:A6:BE:7C:E8:16:00:78:E8:5A:2C:7B:55:9F:36:AE:D0:B6:96:C0:
9D:FA:4E:18:AA:70:4B:3B:94:72:FB:83:DE:8A:D9:F3:80:1E:85:67:
E3:EC:6F:64:CF:B1:B4:34:88:36:1B:19:ED:28:A1:D5:F4:0A:10:9B:
AF:F6:CD:90:36:1D:49:DA:FC:D5:71:71:52:64:F9:6E:26:83:C9:60:
B4:65:15:AC:53:A8:FA:2A:71:CE:12:1B:AB:A1:2C:7D:01:B4:45:AA:
F7:8C:AF:2C:C9:B8:DA:FD:4A:13:AB:A0:94:FE:51:4F:44:41:AF:AB:
95:63:32:32:B7:97:CD:97:F4:72:1E:FE:8B:C4:43:B1:0C:C0:B7:EC:
C1:95:DE:75:0B:65:15:A3:60:9A:B8:BB:2C:90:3A:01:3F:18:27:15:
FC:99:8D:01:E0:5C:B0:B4:78:C4:D7:C6:69:CE:88:78:22:4C:4F:66:
6A:F0:C7:60:3D:F8:14:AC:1E:14:A9:81:AA:DB:82:09:26:49:45:7E:
52:E6:09:02:D6:30:13:14:9E:3D:57:EA:E0:74:8C:14:C9:18:DF:38:
B6:48:66:B2:89:E4:CE:13:33:B8:67:3F:FD:49:41:AD:8E:26:B1:65:
8B:D6:8A:57:E1:D9:7C:98:49:06:D4:7D:CD:0E:D5:57:59:2E:09:B0:
92:61:79:4E:4F:67:EA:2C:AA:D0:C9:E4:7C:B4:A6:50:C6:00:A6:9D:
C6:3D:6D:C4:7B:CF:6C:1D:AB:51:D3:F2:29:D9:B9:A7:98:EA:95:1E:
1D:15:8A:FD:F9:EB:C2:4E:33:D4:E8:97:DF:D0:BE:0E:1A:45:98:AD:
0B:6E:36:B7:91:3F:8F:0C:40:E8:C1:0F:16:F0:60:F1:7E:D3:03:CD:
6D:5A:A7:92:DC:6C:80:F6:40:AE:9B:51:26:58:3A:B3:D7:18:83:EC:
F8:AD:FD:06:84:1D:9E:AA:CE:07:AD:CF
```

2.3. Desafío 1

Seguridad del Kernel: Firma de Módulos y Prevención de Rootkits

Para mitigar riesgos de seguridad y evitar la persistencia de *rootkits* (programas maliciosos que suelen inyectar módulos no autorizados en el núcleo para interceptar llamadas al sistema, ocultar procesos o escalar privilegios), es fundamental restringir la carga de módulos. Esto se logra mediante los siguientes mecanismos de autenticación:

- **Arranque Seguro UEFI (Secure Boot):** Cuando esta característica está habilitada a nivel de *hardware*, el sistema operativo exige estrictamente que **todos los módulos del kernel** estén firmados criptográficamente. El núcleo autentica cada módulo comprobando su firma contra las claves públicas X.509 de confianza almacenadas en los llaveros del sistema. Si un módulo (como el de un *rootkit*) carece de una firma válida, su carga es bloqueada de inmediato.
- **Parámetro `module.sig_enforce`:** Si el entorno no soporta o tiene desactivado el *Secure Boot*, el administrador puede forzar por *software* la verificación de firmas añadiendo este parámetro en la línea de comandos del kernel durante el arranque. Esto garantiza que ningún módulo sin firmar pueda ser insertado mediante `insmod` o `modprobe`.
- **Firmado de módulos propios:** Para cargar módulos legítimos (desarrollados por el usuario o de terceros), es obligatorio generar un par de claves (pública/privada), firmar el binario `.ko` utilizando la utilidad `sign-file` con la clave privada, e inscribir la clave pública en el sistema para que sea reconocida como confiable.

Al aplicar estas políticas, se cierra uno de los vectores de ataque más críticos en sistemas Linux, asegurando que solo el código explícitamente autorizado y verificado pueda interactuar con el espacio del kernel.

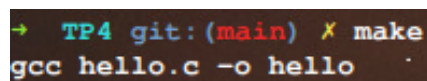
Empaquetado de un "Hello World" mediante Checkinstall

Checkinstall es una utilidad que sustituye el comando tradicional `sudo make install`. En lugar de dispersar archivos directamente en el sistema, intercepta la instalación para generar un paquete nativo (`.deb`, `.rpm` o `.tgz`). De esta manera, centraliza la administración del software compilado desde el código fuente, garantizando su trazabilidad y permitiendo una actualización o desinstalación limpia mediante el gestor de paquetes de la distribución.

Ahora bien, para proceder con el empaquetado de un `hello world`, primero se procede con la instalación de `checkinstall`:

```
1 sudo checkinstall
```

Luego se compila el `hello world`:



```
→ TP4 git:(main) ✗ make  
gcc hello.c -o hello
```

Como puede verse, se ha generado un ejecutable bajo el nombre `hello`.

Por otro lado, el archivo `makefile` utilizado para la compilación tiene el siguiente contenido:

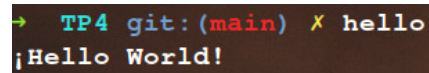
```
1 TARGET = hello  
2  
3 all:  
4     gcc hello.c -o $(TARGET)  
5  
6 install:  
7     install -m 0755 $(TARGET) /usr/local/bin/  
8  
9 clean:  
10    rm -f $(TARGET)
```

Posteriormente, al ejecutar el comando `sudo checkinstall`, el programa `hello` queda empaquetado e integrado formalmente en el sistema operativo. A diferencia de una ejecución local, el binario ahora es gestionado por el administrador de paquetes, lo que permite invocarlo de manera global.

Podemos verificar su correcta instalación ejecutando simplemente:

`hello`

desde cualquier terminal. Observaremos que el sistema lo reconoce y ejecuta como un comando nativo más del sistema:



```
→ TP4 git:(main) ✕ hello
¡Hello World!
```

2.4. Desafío 2

2.4.1. ¿Qué funciones tiene disponible un programa y un módulo?

La siguiente tabla detalla las equivalencias y funciones disponibles para un programa estándar frente a un módulo del kernel, según los conceptos de gestión de memoria, E/S y comunicación.

Categoría	Programa (Mundo de Usuario)	Módulo (Mundo del Kernel)
Salida/Log	<code>printf()</code> : Escribe en la salida estándar.	<code>printk()</code> : Registra mensajes en el buffer circular del kernel.
Memoria	<code>malloc()</code> / <code>free()</code> : Asignación dinámica en el heap del proceso.	<code>kmalloc()</code> / <code>kfree()</code> : Asignación de memoria físicamente contigua en el kernel.
Archivos / E/S	<code>fopen()</code> : Función de alto nivel de <code>stdio.h</code> .	APIs VFS : El kernel usa el Virtual File System internamente.
Acceso a Datos	<code>read()</code> / <code>write()</code> : Llamadas al sistema para leer/escribir buffers.	<code>copy_from_user()</code> / <code>copy_to_user()</code> : Copia segura de datos entre usuario y kernel.
Pausa	<code>sleep()</code> : Suspende el proceso por un tiempo determinado.	<code>msleep()</code> : Pone la tarea actual en estado de espera (no bloqueante para el CPU).
Contexto	Utiliza servicios del kernel mediante interrupciones de software.	Ejecuta código directamente como parte del sistema operativo.

Cuadro 1: Mapeo de instrucciones equivalentes y APIs según el contexto de ejecución.

2.4.2. Espacio de Usuario vs. Espacio del Kernel

Espacio de Usuario (User Space)

- **Definición:** Entorno de ejecución restringido donde operan las aplicaciones y programas estándar.
- **Privilegios:** Limitados. Los procesos aquí no pueden acceder directamente al hardware ni a la memoria de otros procesos.
- **Interacción:** Para realizar tareas de bajo nivel (como E/S de archivos), deben usar *llamadas al sistema* (System Calls) para comunicarse con el núcleo. Estas llamadas pueden rastrearse con herramientas como `strace`.

Espacio del Kernel (Kernel Space)

- **Definición:** Entorno privilegiado reservado para el núcleo del sistema operativo y sus extensiones dinámicas (módulos del kernel).
- **Privilegios:** Máximos (Modo Supervisor). Tiene acceso directo e irrestricto a todo el hardware, la memoria y los recursos del sistema.
- **Seguridad e Impacto:** Un fallo en este espacio compromete todo el sistema (*Kernel Panic*). Por seguridad, distribuciones empresariales requieren que los módulos estén firmados criptográficamente para cargarse en sistemas con *Secure Boot* activo.

2.4.3. Espacio de datos

El espacio de datos es la región de memoria reservada durante la ejecución de un código para almacenar variables, estado y estructuras dinámicas, siendo el área de trabajo esencial para que la CPU procese información. Su correcta gestión es vital para evitar el agotamiento de recursos o accesos inválidos.

Existen diferencias críticas según su contexto de ejecución. El espacio de datos de usuario es privado y está estrictamente aislado para cada proceso; un acceso indebido a la memoria aquí solo provoca la terminación de esa aplicación específica (*Segmentation Fault*). En contraste, el espacio de datos del kernel es un entorno compartido por el núcleo y todos sus módulos. Al carecer de aislamiento interno, un manejo incorrecto de la memoria corrompe todo el sistema operativo, desencadenando un fallo fatal (*Kernel Panic*).

2.4.4. Drivers y Gestión de Periféricos en /dev

Los **drivers** actúan como la capa de abstracción necesaria para que el kernel se comunique con el hardware. En Linux, la gestión de periféricos se centraliza en el directorio `/dev`, basándose en el principio de que los dispositivos se proyectan como archivos lógicos para facilitar su acceso desde el espacio de usuario.

Manejo de Periféricos en /dev

El directorio `/dev` permite que el sistema operativo gestione periféricos mediante dos mecanismos principales de flujo de datos:

- **Periféricos de Flujo Continuo (Character):** Dispositivos que envían o reciben datos byte a byte, como teclados, ratones o puertos serie (`/dev/tty`). El driver gestiona la entrada en tiempo real sin necesidad de un buffer de almacenamiento intermedio.
- **Periféricos de Almacenamiento (Bloque):** Dispositivos como discos duros o unidades USB (`/dev/sdb`). Aquí, el driver organiza los datos en bloques direccionables, permitiendo al sistema leer o escribir en cualquier posición del periférico de forma no lineal.

2.5. Preguntas para el informe

2.5.1. ¿Qué diferencias se pueden observar entre los dos modinfo?

Campo (<i>Field</i>)	<code>des_generic.ko.zst</code>	<code>mimodulo.ko</code>
filename	Ubicado en la jerarquía estándar del kernel (<code>/kernel/crypto/</code>).	Ubicado en ruta absoluta de usuario (<code>/home/esteban/...</code>).
intree	Y (Indica que es parte oficial del código fuente del kernel).	Campo inexistente (Módulo <i>out-of-tree</i> o externo).
alias	Define múltiples nombres alternativos para carga automática.	Campo inexistente en este módulo.
depends	Indica dependencia crítica del módulo <code>libdes</code> .	Campo vacío; es un módulo autónomo.
vermagic	Identifica la versión exacta del kernel y opciones de compilación.	Coincide con el sistema del usuario.
signer / sig_key	Identifican la clave y el autor de la firma criptográfica.	Campos no disponibles.
signature	Bloque hexadecimal que garantiza la integridad del binario.	Campo no disponible.

Cuadro 2: Desglose comparativo incluyendo estado de integración y seguridad.

La diferencia más importante radica en el contenido de los campos `intree` y `signature`, y a continuación está la justificación:

- **Origen y Soporte (`intree`):** El valor Y en `intree` (como se ve en `des_generic`) confirma que el módulo ha sido desarrollado, probado y aceptado dentro del código fuente oficial de Linux. Los módulos que no tienen este campo se consideran *out-of-tree*. El kernel marca el sistema como *tainted* (contaminado) al cargar módulos externos porque no puede garantizar su estabilidad ni compatibilidad a largo plazo.
- **Seguridad y Autenticidad (`signature`):** La firma digital es el mecanismo de control de acceso. Aunque un módulo sea funcional, si carece de `signature` (como `mimodulo.ko`), el hardware mediante *Secure Boot* impedirá su carga para evitar la ejecución de código malicioso o no autorizado en el modo más privilegiado del procesador.

2.5.2. ¿Qué drivers/módulos están cargados en sus propias PC?

Para este apartado, se utilizó el siguiente comando para obtener la información y guardarla en un archivo de texto para luego comparar resultados con los compañeros:

```
lsmod > TP4/lsmod_esteban.txt
```

Y se obtuvo la siguiente salida (Máquina Virtual de Esteban):

Module	Size	Used by
snd_seq_dummy	12288	0
snd_hrtimer	12288	1
qrtr	53248	2
binfmt_misc	24576	1
nls_iso8859_1	12288	1
intel_rapl_msr	20480	0
intel_rapl_common	53248	1 intel_rapl_msr
snd_intel8x0	53248	1
snd_ac97_codec	196608	1 snd_intel8x0
ac97_bus	12288	1 snd_ac97_codec
intel_uncore_frequency_common		16384 0
intel_pmc_core	122880	0
snd_pcm	192512	2 snd_intel8x0,snd_ac97_codec
pmt_telemetry	16384	1 intel_pmc_core
pmt_class	16384	1 pmt_telemetry
intel_vsec	20480	1 intel_pmc_core
snd_seq_midi	24576	0
snd_seq_midi_event	16384	1 snd_seq_midi
snd_rawmidi	57344	1 snd_seq_midi
snd_seq	122880	9 snd_seq_midi,snd_seq_midi_event,snd_seq_dummy
snd_seq_device	16384	3 snd_seq,snd_seq_midi,snd_rawmidi
i2c_piix4	32768	0
rapl	20480	0
snd_timer	53248	3 snd_seq,snd_hrtimer,snd_pcm
snd	143360	11 snd_seq,snd_seq_device,snd_intel8x0,snd_timer,snd_ac97_codec,snd_pcm
i2c_smbus	20480	1 i2c_piix4
joydev	32768	0
vboxguest	57344	0
soundcore	16384	1 snd
input_leds	12288	0
serio_raw	20480	0
mac_hid	12288	0
sch_fq_codel	24576	2
vmwgfx	466944	1
drm_ttm_helper	16384	2 vmwgfx
ttm	118784	2 vmwgfx,drm_ttm_helper
msr	12288	0
parport_pc	53248	0
ppdev	24576	0
lp	28672	0
parport	73728	3 parport_pc,lp,ppdev
efi_pstore	12288	0
nfnetlink	20480	1
dmi_sysfs	24576	0
ip_tables	32768	0
x_tables	65536	1 ip_tables
autofs4	57344	2
btrfs	2088960	0
blake2b_generic	20480	0
raid10	73728	0

raid456	192512	0
async_raid6_recov	20480	1 raid456
async_memcpy	16384	2 raid456,async_raid6_recov
async_pq	20480	2 raid456,async_raid6_recov
async_xor	16384	3 async_pq,raid456,async_raid6_recov
async_tx	16384	5 async_pq,async_memcpy,async_xor,raid456,async_raid6_recov
xor	20480	2 async_xor,btrfs
raid6_pq	126976	4 async_pq,btrfs,raid456,async_raid6_recov
raid1	61440	0
raid0	24576	0
linear	16384	0
dm_mirror	24576	0
dm_region_hash	24576	1 dm_mirror
dm_log	20480	2 dm_region_hash,dm_mirror
hid_generic	12288	0
usbhid	77824	0
polyval_clmulni	12288	0
polyval_generic	12288	1 polyval_clmulni
hid	262144	2 usbhid,hid_generic
psmouse	217088	0
ghash_clmulni_intel	16384	0
e1000	180224	0
vga16fb	32768	0
sha256_ssse3	32768	0
ahci	49152	2
sha1_ssse3	32768	0
vgastate	20480	1 vga16fb
video	77824	0
pata_acpi	12288	0
libahci	53248	1 ahci
wmi	28672	1 video
aesni_intel	122880	0
crypto_simd	16384	1 aesni_intel
cryptd	24576	2 crypto_simd,ghash_clmulni_intel

A continuación se analizan algunos módulos destacados:

Módulo	Categoría	Descripción y Dependencias
<code>snd_pcm</code>	Sonido	Gestiona el audio digital (PCM). Es utilizado por controladores de hardware como <code>snd_intel8x0</code> .
<code>vboxguest</code>	Virtualización	Driver de las <i>Guest Additions</i> de VirtualBox para integración con el anfitrión.
<code>vmwgfx</code>	Gráficos	Driver de aceleración gráfica para entornos VMware o aceleración SVGA.
<code>btrfs</code>	Sistema de Archivos	Soporte para el sistema de archivos Btrfs.
<code>e1000</code>	Red	Controlador para tarjetas de red Intel Ethernet.
<code>usbhid</code>	Periféricos	Gestiona dispositivos de interfaz humana (teclados, ratones) conectados por USB.
<code>aesni_intel</code>	Criptografía	Aceleración por hardware del algoritmo de cifrado AES en procesadores Intel.
<code>ahci</code>	Almacenamiento	Controlador para la interfaz de controladores de host avanzada.

Cuadro 3: Selección de módulos activos y su rol en el sistema.

De la misma manera se aplicó el mismo comando para las computadoras de José y Matías, y luego de efectuada la comparación se llegó a las siguientes conclusiones:

Componente	Matías (AMD)	José (Intel)	Esteban (VM)
Gráficos	<code>amdgpu</code>	<code>i915</code>	<code>vmwgfx</code>
Virtualización	<code>kvm_amd</code>	<code>kvm_intel</code>	<code>vboxguest</code>
Almacenamiento	<code>nvme</code>	<code>ahci</code>	<code>ahci</code>
Red	<code>rtw89</code> (WiFi)	<code>rtl8723</code> (WiFi)	<code>e1000</code> (Eth)
Sonido	<code>snd_sof_amd</code>	<code>snd_hda_intel</code>	<code>snd_intel8x0</code>

- **Matías:** Hardware físico moderno basado en arquitectura AMD y NVMe.
- **José:** Hardware físico basado en Intel con almacenamiento SATA.
- **Esteban:** Sistema virtualizado (VirtualBox) con controladores emulados.

2.5.3. ¿Cuáles módulos no están cargados pero están disponibles?

Si se quiere ver las categorías de módulos principales (entre los disponibles), se debe acceder a la siguiente ruta:

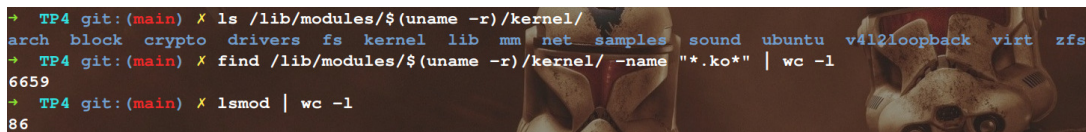
```
ls /lib/modules/$(uname -r)/kernel/
```

Y si se quiere contar el total de módulos disponibles:

```
find /lib/modules/$(uname -r)/kernel/ -name "*.ko*" | wc -l
```

Y si se quiere contar la cantidad de módulos cargados actualmente:

```
lsmod | wc -l
```



```
TP4 git:(main) X ls /lib/modules/$(uname -r)/kernel/
arch block crypto drivers fs kernel lib mm net samples sound ubuntu v4l2loopback virt zfs
TP4 git:(main) X find /lib/modules/$(uname -r)/kernel/ -name "*.ko*" | wc -l
6659
TP4 git:(main) X lsmod | wc -l
86
```

Esta estructura refleja la eficiencia del diseño modular, permitiendo que el kernel mantenga una huella de memoria reducida al cargar dinámicamente solo los controladores y servicios en uso.

¿Qué pasa cuando el driver de un dispositivo no está disponible?

Cuando el kernel de Linux no dispone de un controlador (*driver*) específico para un hardware detectado, se producen las siguientes consecuencias técnicas:

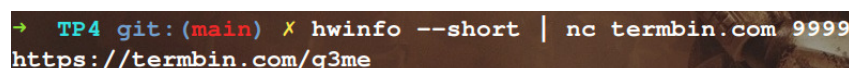
- **Inoperatividad del Hardware:** El periférico permanece en un estado inactivo. Aunque el bus (PCI, USB, etc.) detecte la presencia física del dispositivo, el kernel no sabe cómo inicializarlo ni enviarle comandos.
- **Ausencia de Nodo en /dev:** Al no haber un driver que reclame el dispositivo, no se crea el archivo especial (*device node*) en el directorio /dev. Esto impide que las aplicaciones de usuario puedan abrirlo o interactuar con él.
- **Carga de Drivers Genéricos:** En ciertos casos, el sistema utiliza controladores genéricos. Esto permite una funcionalidad básica y limitada, pero sin acceso a características avanzadas como aceleración gráfica o ahorro de energía.
- **Detección sin Asignación:** Herramientas de diagnóstico como `lspci` o `lsusb` mostrarán el dispositivo con la etiqueta "*Unassigned class*" o sin un "*Kernel driver in use*", indicando que el hardware es reconocido pero no está gestionado.

2.5.4. Correr hwinfo en una pc real con hw real y agregar la url de la información de hw en el reporte.

Si ejecutamos el comando:

```
hwinfo --short | nc termbin.com 9999
```

Observamos el siguiente resultado:



```
TP4 git:(main) X hwinfo --short | nc termbin.com 9999
https://termbin.com/g3me
```

Donde el link solicitado es el siguiente:

<https://termbin.com/g3me>

2.5.5. ¿Qué diferencia existe entre un módulo y un programa?

La distinción principal se basa en el modelo de ejecución jerárquico del sistema operativo.

Característica	Programa (User Space)	Módulo (Kernel Space)
Espacio	User Space: Ejecución en modo restringido (Anillo 3).	Kernel Space: Ejecución en modo privilegiado (Anillo 0).
Privilegios	Acceso limitado al hardware. Debe solicitar recursos mediante <i>System Calls</i> .	Acceso total y directo al hardware y a las instrucciones críticas del CPU.
Memoria	Utiliza memoria virtual protegida. Si falla, el proceso termina (<i>Seg-fault</i>).	Comparte el espacio de memoria del kernel. Si falla, puede provocar un <i>Kernel Panic</i> .
Ejecución	Tiene un punto de entrada definido (main) y ejecución secuencial.	Es un manejador de eventos. Se carga y espera ser llamado por el kernel (init_module).

Cuadro 4: Comparativa técnica entre espacio de usuario y espacio de núcleo.

2.5.6. ¿Cómo puede ver una lista de las llamadas al sistema que realiza un simple hello-world en c?

De acuerdo con la bibliografía seleccionada para la presente práctica, la metodología más eficiente para auditar la interacción entre el *User Space* y el *Kernel Space* consiste en el uso de la herramienta **strace**. Esta utilidad permite interceptar y registrar de forma pormenorizada las llamadas al sistema (*system calls*) que un proceso ejecuta durante su ciclo de vida.

Para ilustrar este mecanismo, se toma como caso de estudio el binario derivado de `hello.c`:

```
$ gcc hello.c -o hello
$ strace ./hello
```

[illegible]

La instrucción crítica en este análisis es `write(1, "¡Hello World!\n", 15)`, la cual representa el instante preciso en que el programa solicita al núcleo la salida del mensaje por pantalla a través del descriptor de archivo estándar.

2.5.7. ¿Qué es un segmentation fault? ¿Cómo lo maneja el kernel y como lo hace un programa?

Un **segmentation fault** ocurre cuando un proceso intenta acceder a una dirección de memoria inválida o protegida, violando las políticas de la MMU (*Memory Management Unit*).

Entidad	Manejo en User Space	Manejo en Kernel Space
Kernel	Envía señal SIGSEGV al proceso. El sistema permanece estable.	Produce un Kernel Panic . El sistema se detiene para evitar daños.
Programa	Termina su ejecución (<i>default</i>) o intenta una limpieza vía <i>signal handler</i> .	El código del módulo falla críticamente; no hay recuperación posible.

Cuadro 5: Diferencia de impacto entre programas y módulos.

2.5.8. Firma de un módulo de Kernel

Primero creamos una clave privada y un certificado público válidos por muchos años:

```
TP4 git:(main) X openssl req -new -x509 -newkey rsa:2048 -keyout MOK.priv -outform DER -out MOK.der -nodes -days 36500 -subj "/CN=Firma_Modulos_TP/"
```

Luego importamos la clave pública al sistema:

```
TP4 git:(main) X sudo mokutil --import MOK.der
[sudo] contraseña para esteban:
EFI variables are not supported on this system
```

Al intentar registrar la clave pública con `mokutil`, el sistema devolvió el error: *"EFI variables are not supported on this system"*.

- **Diagnóstico:** La máquina virtual está configurada para arrancar mediante BIOS Legacy en lugar de UEFI.
- **Impacto:** Al carecer de UEFI, el entorno no soporta *Secure Boot*. Por lo tanto, el kernel no bloqueará la carga de módulos no firmados.
- **Siguiente paso:** Aunque el sistema no exija la validación criptográfica, se procederá a firmar el archivo `.ko` manualmente para demostrar el proceso de inyección de la firma a nivel de archivo.

Entonces, a continuación firmamos el archivo:

```
module git:(main) X sudo /usr/src/linux-headers-$(uname -r)/scripts/sign-file sha256 ./MOK.priv ./MOK.der mimodulo.ko
```

Y verificamos la firma:

```
module git:(main) X modinfo mimodulo.ko | grep signer
signer:      Firma_Modulos_TP
```

Y si queremos obtener los detalles de la firma:

```
+ module git:(main) * modinfo mimodulo.ko | tail -n 20
name:          mimodulo
retpoline:     Y
vermagic:      6.14.0-37-generic SMP preempt mod_unload modversions
sig_id:        PKCS#7
signer:        Firma_Modulos_TP
sig_key:       6E:0E:86:28:BA:DA:84:3B:6D:98:2E:49:61:64:B9:F2:2B:DE:69:7D
sig_hashalgo:  sha256
signature:     38:DC:59:68:1B:71:38:EF:8E:20:07:3F:66:5B:FD:3E:AB:D7:D6:87:
04:E1:42:94:59:CB:0C:58:1B:62:ED:CE:2D:AA:BA:41:9F:2C:C7:9D:
62:78:74:F2:68:92:8D:CD:99:00:0A:5B:12:70:F2:82:08:09:2C:A4:
04:12:B4:23:53:2E:77:0D:AF:5C:EE:90:A2:AE:D9:5A:DE:0C:48:AD:
8D:7A:F0:4E:39:EE:FB:90:34:5A:7D:9F:01:C5:08:3F:C3:E0:D4:51:
1C:28:EB:13:79:F8:FE:BC:56:1F:BF:EA:78:0A:7E:83:46:F8:9A:84:
E1:FC:1D:75:3C:9E:9F:5D:B3:D9:72:34:B9:A8:10:4A:8B:9C:75:39:
D9:D0:81:B8:7D:7E:4C:E1:9B:AD:03:E7:E6:5A:CD:D9:02:0B:76:B6:
67:81:01:2D:CF:6B:20:55:14:0D:D6:4B:01:B8:06:DB:B1:2B:6E:FC:
31:BB:FD:2F:68:C1:C8:2B:5F:1A:61:F4:F2:BD:E6:02:A1:65:E1:C1:
E1:F1:7C:1B:C0:32:34:A3:0B:0B:AE:3A:90:66:1C:72:9F:26:F6:6A:
7E:20:5C:AE:23:2D:49:DA:94:B1:52:2A:12:BF:4E:C0:98:DC:6D:53:
F2:5E:C9:64:F6:B0:FA:4B:21:FB:F0:02:A0:05:82:A9
```

2.5.9. Agregar evidencia de la compilación, carga y descarga de su propio módulo imprimiendo el nombre del equipo en los registros del kernel

En primera instancia, se generó un archivo `electro.c` con el siguiente contenido:

```
1 #include <linux/module.h>
2 #include <linux/kernel.h>
3 #include <linux/init.h>
4
5 MODULE_LICENSE("GPL");
6 MODULE_AUTHOR("Equipo Electro");
7 MODULE_DESCRIPTION("Modulo basico para TP");
8
9 static int __init electro_init(void) {
10     printk(KERN_INFO "Equipo Electro: Modulo cargado exitosamente.\n");
11     return 0;
12 }
13
14 static void __exit electro_exit(void) {
15     printk(KERN_INFO "Equipo Electro: Modulo descargado.\n");
16 }
17
18 module_init(electro_init);
19 module_exit(electro_exit);
```

Y un archivo Makefile con el siguiente contenido:

```
1 obj-m += electro.o
2
3 all:
4     make -C /lib/modules/$(shell uname -r)/build M=$(PWD) modules
5
6 clean:
7     make -C /lib/modules/$(shell uname -r)/build M=$(PWD) clean
```

Y a continuación se ejecutaron los siguientes comandos:

1. Compilar

make

2. Cargar el módulo

sudo insmod electro.ko

3. Ver el registro del kernel (Evidencia de carga)

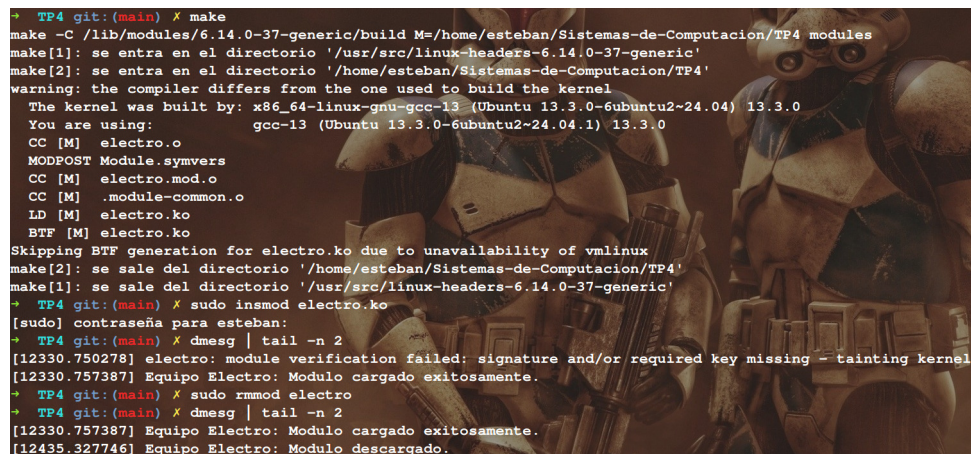
dmesg | tail -n 2

4. Descargar el módulo

sudo rmmod electro

5. Ver el registro del kernel (Evidencia de descarga)

dmesg | tail -n 2



```
TP4 git:(main) X make
make -C /lib/modules/6.14.0-37-generic/build M=/home/esteban/Sistemas-de-Computacion/TP4 modules
make[1]: se entra en el directorio '/usr/src/linux-headers-6.14.0-37-generic'
make[2]: se entra en el directorio '/home/esteban/Sistemas-de-Computacion/TP4'
warning: the compiler differs from the one used to build the kernel
The kernel was built by: x86_64-linux-gnu-gcc-13 (Ubuntu 13.3.0-6ubuntu2~24.04) 13.3.0
You are using: gcc-13 (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
CC [M] electro.o
MODPOST Module.symvers
CC [M] electro.mod.o
CC [M] .module-common.o
LD [M] electro.ko
BTF [M] electro.ko
Skipping BTF generation for electro.ko due to unavailability of vmlinux
make[2]: se sale del directorio '/home/esteban/Sistemas-de-Computacion/TP4'
make[1]: se sale del directorio '/usr/src/linux-headers-6.14.0-37-generic'
TP4 git:(main) X sudo insmod electro.ko
[sudo] contraseña para esteban:
TP4 git:(main) X dmesg | tail -n 2
[12330.750278] electro: module verification failed: signature and/or required key missing - tainting kernel
[12330.757387] Equipo Electro: Modulo cargado exitosamente.
TP4 git:(main) X sudo rmmod electro
TP4 git:(main) X dmesg | tail -n 2
[12330.757387] Equipo Electro: Modulo cargado exitosamente.
[12435.327746] Equipo Electro: Modulo descargado.
```

El mensaje que indica "*tainting kernel*" aparece porque el módulo se compiló localmente y carece de una firma digital válida. Al no poder verificar criptográficamente la procedencia del código, el sistema operativo deja una constancia en sus registros de que ahora está ejecutando código de terceros no oficial (lo marca como "manchado"). Al no tener *Secure Boot* activado, el kernel solo emite esta advertencia, pero permite la carga del módulo sin problemas.

2.5.10. ¿Que pasa si mi compañero con secure boot habilitado intenta cargar un módulo firmado por mi?

El rechazo del módulo ocurre debido a que la cadena de confianza criptográfica está incompleta en el sistema de destino:

- **Llave Privada:** El módulo fue sellado con mi llave privada personal durante el proceso de firma.
- **Llave Pública (MOK.der):** El kernel del compañero intenta verificar la firma, pero no encuentra mi llave pública en su base de datos de confianza local.

Entonces, aunque el archivo está firmado, el sistema receptor no tiene la "llave" necesaria para validar dicha firma, resultando en el error: **Required key not available**.

2.5.11. Análisis de la nota propuesta sobre Secure Boot

¿Cuál fue la consecuencia principal del parche de Microsoft sobre GRUB en sistemas con arranque dual (Linux y Windows)?

La consecuencia principal fue el **bloqueo del inicio de las distribuciones Linux**. Microsoft aplicó una actualización de la política *Secure Boot Advanced Targeting* (SBAT) para revocar cargadores de arranque (GRUB) vulnerables. Debido a una implementación defectuosa o desactualizada en varias distribuciones, el sistema detectaba una "Violación de la Política de Seguridad", impidiendo que el usuario accediera a Linux en equipos con Windows y Secure Boot activo.

¿Qué implicancia tiene desactivar Secure Boot como solución al problema descrito en el artículo?

Desactivar *Secure Boot* funciona como solución temporal para recuperar el acceso al sistema, pero conlleva una **degradación crítica de la seguridad**. Al hacerlo, se rompe la cadena de confianza, permitiendo que el hardware ejecute cualquier código de arranque sin verificar. Esto deja al equipo vulnerable ante *bootkits* y *rootkits* que pueden infectar el sistema antes de que el propio sistema operativo y el antivirus se carguen.

¿Cuál es el propósito principal del Secure Boot en el proceso de arranque de un sistema?

El propósito principal es **garantizar la integridad del proceso de arranque**. Asegura que el firmware de la computadora (*UEFI*) ejecute únicamente software (cargadores de arranque, kernels y controladores) que contenga una firma digital válida y reconocida por una autoridad de confianza. Su objetivo final es evitar que software malicioso no autorizado tome el control del equipo durante el encendido.